

Assignment 2: Caribbean Wildlife Conservation Tracker

Course: COMP2603 Object Oriented Programming 1

Weight: 10%

Given: Week 4

Due: End of Week 6

Covers: Weeks 1 to 4 (classes, constructors, static, ArrayList, encapsulation, inheritance, abstract classes, interfaces, polymorphism, dynamic dispatch, instanceof, Swing GUI)

The story

The Caribbean Wildlife Trust manages sanctuaries across the islands. Animals are tracked, relocated between sanctuaries, and monitored through sighting logs. Rangers record every sighting. When an animal needs to move to a different island, the system must check whether the species is eligible for relocation, calculate the cost, and update the records. Some species cannot be relocated at all.

You will build the core data model (an abstract class hierarchy with two interfaces), the sanctuary management logic, a console demo that exercises every feature, and a small Swing GUI for searching and viewing animals.

What the system needs to do

When you are finished, your system must be able to:

1. Create animals of three types (Bird, Reptile, Marine) with auto-assigned IDs.
2. Enforce valid health statuses and reject bad input.
3. Log sightings for any trackable animal and retrieve the count and most recent sighting.
4. Determine which animals can be relocated and calculate relocation costs.
5. Add and remove animals from sanctuaries, respecting capacity and island constraints.
6. Filter animals by type, find the most expensive to feed, and list relocatable animals.
7. Transfer an animal between sanctuaries, failing gracefully if the animal is not relocatable.
8. Search animals in a Swing GUI by name, type, and health status.

What you will learn

- Design an abstract base class with static ID generation and input validation.
- Define interfaces and implement them in concrete subclasses.
- Use `super(...)` to chain constructors through an inheritance hierarchy.
- Override abstract methods and rely on dynamic dispatch for polymorphic behaviour.
- Use `instanceof` to check interface conformance at runtime.
- Store mixed types in a polymorphic `ArrayList<Animal>` collection.

- Build a Swing GUI with JFrame, BorderLayout, FlowLayout, JTextField, JComboBox, JCheckBox, JTextArea, JScrollPane, ActionListener, and KeyListener.
- Separate model logic from view logic (Model-View separation).

Files

```
Trackable.java      // interface for sighting logs
Relocatable.java    // interface for inter-island transfers
Animal.java         // abstract base class
Bird.java           // extends Animal, implements Trackable &
                    Relocatable
Reptile.java        // extends Animal, implements Trackable
Marine.java         // extends Animal, implements Trackable &
                    Relocatable
Sanctuary.java      // holds animals, manages operations
Driver.java         // scripted console demo (no GUI)
SanctuaryGUI.java   // Swing search interface
```

Submission

Do not upload any code to myElearning. Push your nine .java files to a **public GitHub repository**, then submit a single plain text file named `<id_number>_a2.txt` via myElearning containing your repository URL and the commit hash of the commit you want graded, for example:

```
Repository: https://github.com/jane-doe/comp2603-a2
Commit:     7d1c9f02a3e5b8746c0d1e2f3a4b5c6d7e8f9a0b
```

Run `git rev-parse HEAD` after you `git push` to get the hash. See the assignment guidelines document for the full step-by-step instructions and the submission rules.

Class hierarchy



```
|           |
+-----+--- both implement Relocatable
```

Bird and Marine implement both Trackable and Relocatable. Reptile implements only Trackable. Reptiles cannot be relocated in this system.

Milestones

M1. Animal abstract class: fields, constructor, toString

Open `Animal.java`. Declare:

- A private static `int` `nextId` field, initialized to 1.
- Private instance fields: `animalId` (`int`), `species` (`String`), `nickname` (`String`), `island` (`String`), `weightKg` (`double`), `healthStatus` (`String`).

Write the constructor `Animal(String species, String nickname, String island, double weightKg, String healthStatus)`:

- Validate that `species`, `nickname`, and `island` are not null and not empty (after trimming). Throw `IllegalArgumentException` if invalid.
- Validate that `weightKg > 0`. Throw `IllegalArgumentException` if not.
- Validate that `healthStatus` is exactly one of "Healthy", "Injured", or "Critical". Throw `IllegalArgumentException` otherwise.
- Assign `animalId = nextId` then increment `nextId`.

Write getters for all six fields: `getAnimalId()`, `getSpecies()`, `getNickname()`, `getIsland()`, `getWeightKg()`, `getHealthStatus()`.

Write `updateHealth(String newStatus)` that validates the new status using the same three allowed values, then updates the field.

Write `toString()` returning the format:

```
#001 Scarlet Ibis 'Ruby' (Trinidad) [Bird] 0.35 kg - Healthy
```

Use `String.format("#%03d %s '%s' (%s) [%s] %.2f kg - %s", ...)`. Note that `getType()` is called here, which is abstract. This works because `toString()` will only ever be called on a concrete subclass.

Checkpoint: The file should not compile yet because `getType()` and `getDailyFoodCostTTD()` have no bodies. That is correct. Move on to M2.

M2. Subclasses: Bird, Reptile, Marine

Declare the two abstract methods in `Animal.java`:

```
public abstract String getType();
public abstract double getDailyFoodCostTTD();
```

Also add `public void setIsland(String island)` in `Animal`. The `Relocatable` implementations will need it.

Open Bird.java. Make it extends Animal:

- Private fields: wingspanCm (double), canFly (boolean).
- Constructor: Bird(String species, String nickname, String island, double weightKg, String healthStatus, double wingspanCm, boolean canFly). Call super(...) with the first five parameters, then set the Bird-specific fields.
- getType() returns "Bird".
- getDailyFoodCostTTD() returns 15.0 + getWeightKg() * 50.0.
- Getters: getWingspanCm(), canFly().

Open Reptile.java. Make it extends Animal:

- Private fields: isVenomous (boolean), lengthCm (double).
- Constructor: Reptile(String species, String nickname, String island, double weightKg, String healthStatus, boolean isVenomous, double lengthCm).
- getType() returns "Reptile".
- getDailyFoodCostTTD() returns 25.0 + getWeightKg() * 2.0.
- Getters: isVenomous(), getLengthCm().

Open Marine.java. Make it extends Animal:

- Private fields: maxDepthM (double), tankSizeLitres (int).
- Constructor: Marine(String species, String nickname, String island, double weightKg, String healthStatus, double maxDepthM, int tankSizeLitres).
- getType() returns "Marine".
- getDailyFoodCostTTD() returns 50.0 + getWeightKg() * 3.0.
- Getters: getMaxDepthM(), getTankSizeLitres().

Checkpoint: You should be able to write new Bird("Scarlet Ibis", "Ruby", "Trinidad", 0.35, "Healthy", 60.0, true) and call toString() on it. The output should be #001 Scarlet Ibis 'Ruby' (Trinidad) [Bird] 0.35 kg - Healthy. Compile all files and test this in a temporary main method.

M3. Interfaces: Trackable and Relocatable

Open Trackable.java. Declare three method signatures:

```
public interface Trackable {
    void logSighting(String date, String location);
    int getSightingCount();
    String getLastSighting();
}
```

Open Relocatable.java. Declare three method signatures:

```
public interface Relocatable {
    boolean canRelocateTo(String targetIsland);
    double getRelocationCost();
}
```

```
void relocateTo(String island);  
}
```

Update the class declarations:

- Bird **becomes** public class Bird extends Animal implements Trackable, Relocatable
- Reptile **becomes** public class Reptile extends Animal implements Trackable
- Marine **becomes** public class Marine extends Animal implements Trackable, Relocatable

Do not implement the interface methods yet. The code will not compile until M4 and M6 are done.

M4. Implement Trackable methods

Add a private `ArrayList<String> sightings` field to `Animal.java`. Initialize it in the constructor as an empty list. Add a protected getter `getSightings()` that returns this list.

All three subclasses implement `Trackable` identically:

- `logSighting(String date, String location)`: append `date + " at " + location` to the sightings list.
- `getSightingCount()`: return the size of the sightings list.
- `getLastSighting()`: return the last element of the sightings list, or "No sightings recorded" if the list is empty.

Implement these three methods in `Bird`, `Reptile`, and `Marine`.

Checkpoint: Create a `Bird`, call `logSighting("2026-06-10", "Caroni Swamp")`, then verify `getSightingCount()` returns 1 and `getLastSighting()` returns "2026-06-10 at Caroni Swamp". On a brand-new animal with no sightings, `getLastSighting()` should return "No sightings recorded".

M5. Sanctuary: core operations

Open `Sanctuary.java`. Declare:

- Private fields: `name (String)`, `island (String)`, `capacity (int)`, `animals (ArrayList<Animal>)`.
- Constructor `Sanctuary(String name, String island, int capacity)` initializes all fields. The list starts empty.

Write getters for `name`, `island`, `capacity`, and `animals`.

Write `addAnimal(Animal a)`:

- Return false if `a` is null.
- Return false if the sanctuary is at capacity.
- Return false if `a.getIsland()` does not equal the sanctuary's island.
- Otherwise add and return true.

Write `removeAnimal(int animalId)`:

- Loop through the list, find the animal with the matching ID, remove it, and return it.
- Return `null` if no match is found.

Write `getAnimalCount()` returning the list size.

Write `printRoster()`: loop through animals and print each one's `toString()` indented by two spaces.

Write `toString()` in this format:

```
Caroni Bird Sanctuary (Trinidad) [6/20 animals]
```

Also implement `equals` and `hashCode` in `Animal.java`:

- Two animals are equal if they have the same `animalId`.
- `hashCode` returns `Integer.hashCode(animalId)`.

Checkpoint: Create a Sanctuary, add a few animals, call `printRoster()`, and verify the output. Try adding an animal whose island does not match the sanctuary: it should be rejected.

M6. Implement Relocatable methods

In `Bird.java`:

- `canRelocateTo(String targetIsland)`: always returns `true`. Birds can be transported anywhere.
- `getRelocationCost()`: returns `500.0 + getWeightKg() * 100.0`.
- `relocateTo(String island)`: calls `setIsland(island)`.

In `Marine.java`:

- `canRelocateTo(String targetIsland)`: returns `true` only if `targetIsland` is not the animal's current island.
- `getRelocationCost()`: returns `2000.0 + getTankSizeLitres() * 5.0`.
- `relocateTo(String island)`: calls `setIsland(island)`.

Reptile does NOT implement Relocatable. This is deliberate. Reptiles cannot be transferred between islands in this system.

Checkpoint: Create a Bird and a Marine. Call `canRelocateTo("Jamaica")` on both. The Bird should return `true`. The Marine should return `true` if its current island is not Jamaica, `false` if it is already on Jamaica.

M7. Sanctuary: query methods

Write `getAnimalsOfType(String type)`:

- Return a new `ArrayList<Animal>` containing every animal where `getType().equals(type)` is `true`.

Write `getDailyFoodBudget()`:

- Sum `getDailyFoodCostTTD()` for every animal. Return the result rounded to two decimal places. Use `Math.round(total * 100.0) / 100.0`.

Write `getMostExpensiveAnimal()`:

- Return the animal with the highest `getDailyFoodCostTTD()`, or null if the sanctuary is empty.

Checkpoint: With the Caroni sanctuary holding 3 Birds (0.35, 0.40, 0.25 kg), 2 Reptiles (45.0, 30.0 kg), and 1 Marine (500.0 kg), `getDailyFoodBudget()` should return 1845.00.

`getMostExpensiveAnimal()` should be Atlas (the Marine) at \$1550.00/day.

M8. Sanctuary: transfer logic

Write `getRelocatableAnimals()`:

- Return a new `ArrayList<Animal>` containing every animal that is instanceof `Relocatable`.

Write `transferAnimal(int animalId, Sanctuary target)`:

9. Call `removeAnimal(animalId)`. If null, return false.

10. Check if the animal instanceof `Relocatable`. If it is NOT:

- Re-add the animal to this sanctuary (put it back).
- Return false.

11. Cast to `Relocatable` and call `relocateTo(target.getIsland())`.

12. Call `target.addAnimal(animal)`. If that fails (returns false):

- Call `relocateTo(this.getIsland())` to undo the island change.
- Re-add the animal to this sanctuary.
- Return false.

13. Return true.

Checkpoint: Transfer Atlas (Marine) from Caroni to Blue Lagoon. It should succeed. Atlas's island should change from Trinidad to Jamaica. Then attempt to transfer Brutus (Reptile) from Caroni to Blue Lagoon. It should fail because Reptile does not implement `Relocatable`. Brutus should remain in Caroni with his island unchanged.

M9. Driver: console demo

Open `Driver.java`. Create two sanctuaries and add the following animals.

Caroni Bird Sanctuary (Trinidad, capacity 20):

#	Type	Species	Nickname	Weight	Health	Extra fields
1	Bird	Scarlet Ibis	Ruby	0.35	Healthy	wingspan 60.0, canFly true
2	Bird	Scarlet Ibis	Blaze	0.40	Healthy	wingspan

						58.0, canFly true
3	Bird	Cocrico	Dusty	0.25	Injured	wingspan 30.0, canFly true
4	Reptile	Spectacled Caiman	Brutus	45.0	Healthy	venomous false, length 180.0
5	Reptile	Green Anaconda	Medusa	30.0	Critical	venomous false, length 350.0
6	Marine	Leatherback Turtle	Atlas	500.0	Healthy	depth 1200.0, tank 8000

Blue Lagoon Marine Park (Jamaica, capacity 15):

#	Type	Species	Nickname	Weight	Health	Extra fields
7	Bird	Doctor Bird	Flash	0.01	Healthy	wingspan 12.0, canFly true
8	Marine	Hawksbill Turtle	Shelly	80.0	Injured	depth 50.0, tank 3000
9	Marine	Nurse Shark	Gills	110.0	Healthy	depth 75.0, tank 5000

Print the following output blocks in order. The complete expected output is shown at the end of this document.

14. Print `=== Caroni Bird Sanctuary roster ===` then `caroni.printRoster()`.
15. Print `=== Blue Lagoon Marine Park roster ===` then `blueLagoon.printRoster()`.
16. Print `=== Daily food budgets ===` then each sanctuary's name and budget.
17. Print `=== Birds at Caroni ===` then each bird, indented.
18. Print `=== Relocatable animals at Caroni ===` then each relocatable animal, indented.
19. Print `=== Sighting logs ===`. Log sightings for Ruby and Atlas, then print counts and last sightings.
20. Print `=== Transfer Atlas to Blue Lagoon ===`. Transfer Atlas, print result, then print the Blue Lagoon roster.

21. Print "=== Attempt to transfer Brutus (Reptile) ===". Attempt the transfer, print result.
22. Print "=== Most expensive animal at each sanctuary ===".
23. Print "=== Updated food budgets ===".

M10. SanctuaryGUI: model wiring

Open `SanctuaryGUI.java`. Add a private `Sanctuary` `sanctuary` field.

Write `setModel(Sanctuary s)`:

- Store the sanctuary reference.
- Optionally update the window title to include the sanctuary name.

This separates the model (`Sanctuary`) from the view (the GUI). The GUI does not create animals or sanctuaries. It receives them through `setModel`.

M11. SanctuaryGUI: search logic and listeners

Write the `runSearch()` method:

24. Get the text from the search field. Trim it and convert to lowercase.
25. Get the selected type from the combo box.
26. Get the checkbox state.
27. Loop through the sanctuary's animals. For each animal, apply all three filters:
 - If the search text is non-empty, keep only animals whose species or nickname contains the text (case-insensitive comparison).
 - If the selected type is not "All", keep only animals whose `getType()` matches.
 - If the checkbox is selected, keep only animals whose health status is "Injured" or "Critical".
28. Build a string from the matching animals (one `toString()` per line) and set it in the text area.
29. Update the status label: "No matches", "1 result", or "N results".

Wire the listeners:

- Add an `ActionListener` to the search button that calls `runSearch()`.
- Add a `KeyListener` to the text field. In `keyReleased`, call `runSearch()`. This gives live-as-you-type filtering.

M12. SanctuaryGUI: main method

Write a main method in `SanctuaryGUI.java`:

30. Create the Caroni Bird Sanctuary with the same animals as the `Driver`.
31. Create a `SanctuaryGUI` instance.
32. Call `setModel(sanctuary)`.
33. Call `setVisible(true)`.

Run the GUI. Type "ibis" into the search field. You should see two results (Ruby and Blaze). Select "Bird" from the type dropdown and click Search. You should see three results. Check "Injured/Critical only" and click Search. You should see one result (Dusty).

Expected Driver output

```
=== Caroni Bird Sanctuary roster ===
#001 Scarlet Ibis 'Ruby' (Trinidad) [Bird] 0.35 kg - Healthy
#002 Scarlet Ibis 'Blaze' (Trinidad) [Bird] 0.40 kg - Healthy
#003 Cocrico 'Dusty' (Trinidad) [Bird] 0.25 kg - Injured
#004 Spectacled Caiman 'Brutus' (Trinidad) [Reptile] 45.00 kg -
Healthy
#005 Green Anaconda 'Medusa' (Trinidad) [Reptile] 30.00 kg -
Critical
#006 Leatherback Turtle 'Atlas' (Trinidad) [Marine] 500.00 kg -
Healthy

=== Blue Lagoon Marine Park roster ===
#007 Doctor Bird 'Flash' (Jamaica) [Bird] 0.01 kg - Healthy
#008 Hawksbill Turtle 'Shelly' (Jamaica) [Marine] 80.00 kg -
Injured
#009 Nurse Shark 'Gills' (Jamaica) [Marine] 110.00 kg - Healthy

=== Daily food budgets ===
Caroni Bird Sanctuary: $1845.00 TTD
Blue Lagoon Marine Park: $685.50 TTD

=== Birds at Caroni ===
#001 Scarlet Ibis 'Ruby' (Trinidad) [Bird] 0.35 kg - Healthy
#002 Scarlet Ibis 'Blaze' (Trinidad) [Bird] 0.40 kg - Healthy
#003 Cocrico 'Dusty' (Trinidad) [Bird] 0.25 kg - Injured

=== Relocatable animals at Caroni ===
#001 Scarlet Ibis 'Ruby' (Trinidad) [Bird] 0.35 kg - Healthy
#002 Scarlet Ibis 'Blaze' (Trinidad) [Bird] 0.40 kg - Healthy
#003 Cocrico 'Dusty' (Trinidad) [Bird] 0.25 kg - Injured
#006 Leatherback Turtle 'Atlas' (Trinidad) [Marine] 500.00 kg -
Healthy

=== Sighting logs ===
Ruby sighting count: 2
Ruby last sighting: 2026-06-12 at Nariva Swamp
Atlas sighting count: 1
Atlas last sighting: 2026-06-11 at Matura Beach
Brutus last sighting: No sightings recorded

=== Transfer Atlas to Blue Lagoon ===
Transfer successful: true
```

```

Blue Lagoon Marine Park roster after transfer:
  #007 Doctor Bird 'Flash' (Jamaica) [Bird] 0.01 kg - Healthy
  #008 Hawksbill Turtle 'Shelly' (Jamaica) [Marine] 80.00 kg -
Injured
  #009 Nurse Shark 'Gills' (Jamaica) [Marine] 110.00 kg - Healthy
  #006 Leatherback Turtle 'Atlas' (Jamaica) [Marine] 500.00 kg -
Healthy

=== Attempt to transfer Brutus (Reptile) ===
Transfer successful: false

=== Most expensive animal at each sanctuary ===
Caroni: #004 Spectacled Caiman 'Brutus' (Trinidad) [Reptile] 45.00
kg - Healthy ($115.00 TTD/day)
Blue Lagoon: #006 Leatherback Turtle 'Atlas' (Jamaica) [Marine]
500.00 kg - Healthy ($1550.00 TTD/day)

=== Updated food budgets ===
Caroni Bird Sanctuary: $295.00 TTD
Blue Lagoon Marine Park: $2235.50 TTD

```

Food cost reference table

Animal	Type	Weight	Formula	Cost/day
Ruby	Bird	0.35	$15.0 + 0.35 * 50.0$	\$32.50
Blaze	Bird	0.40	$15.0 + 0.40 * 50.0$	\$35.00
Dusty	Bird	0.25	$15.0 + 0.25 * 50.0$	\$27.50
Brutus	Reptile	45.0	$25.0 + 45.0 * 2.0$	\$115.00
Medusa	Reptile	30.0	$25.0 + 30.0 * 2.0$	\$85.00
Atlas	Marine	500.0	$50.0 + 500.0 * 3.0$	\$1550.00
Flash	Bird	0.01	$15.0 + 0.01 * 50.0$	\$15.50
Shelly	Marine	80.0	$50.0 + 80.0 * 3.0$	\$290.00
Gills	Marine	110.0	$50.0 + 110.0 * 3.0$	\$380.00

Acceptance criteria

- All 9 files compile with `javac *.java`.
- `java Driver` produces exactly the output shown above.
- `new Animal(...)` does not compile (it is abstract).
- `new Bird("Ibis", "Ruby", "Trinidad", -1, "Healthy", 60, true)` throws `IllegalArgumentException`.
- `new Bird("Ibis", "Ruby", "Trinidad", 0.5, "Unknown", 60, true)` throws `IllegalArgumentException`.
- A Sanctuary rejects animals whose island does not match.
- A Sanctuary rejects additions when at capacity.
- `getRelocatableAnimals()` returns Birds and Marines but not Reptiles.
- Transferring a Reptile returns `false` and leaves it in the original sanctuary.
- Transferring a Marine to a different island succeeds and updates the animal's island.
- `getLastSighting()` returns "No sightings recorded" when no sightings have been logged.
- The GUI displays matching animals when text is typed into the search field.
- The GUI filters by type when a specific type is selected in the combo box.
- The GUI filters by health status when the checkbox is checked.
- The status label shows the correct count of matching results.

Common pitfalls

34. **Forgetting `super(...)` in subclass constructors.** The compiler will complain that `Animal` has no zero-argument constructor. Every subclass constructor must call `super(species, nickname, island, weightKg, healthStatus)` as its first statement.
35. **Making `Animal` fields `public` instead of `private`.** The fields must be `private` with getters. Subclasses access them through `getWeightKg()`, `getIsland()`, etc. The only exception is sightings, which needs a protected getter so subclasses can implement `Trackable`.
36. **Not calling `getType()` in `toString()`.** The format string includes `[Bird]`, `[Reptile]`, or `[Marine]`. This must come from `getType()`, not a hardcoded string in `Animal`. Dynamic dispatch makes this work.
37. **Casting before checking with `instanceof`.** In `transferAnimal`, you must check `if (animal instanceof Relocatable)` before casting. Otherwise a `Reptile` will cause a `ClassCastException`.
38. **Forgetting to re-add the animal on transfer failure.** If a `Reptile` fails the `Relocatable` check, it must be put back into the original sanctuary. If you forget, the animal disappears.
39. **Using `==` instead of `.equals()` for String comparison.** Use `getType().equals(type)`, not `getType() == type`. The `==` operator compares references, not values.
40. **Not initializing the sightings list.** If you forget `sightings = new ArrayList<String>()` in the `Animal` constructor, every call to `logSighting` will throw `NullPointerException`.

41. **Making the JTextArea editable.** Call `resultArea.setEditable(false)` so users cannot type into the results area.
42. **Not handling the empty sanctuary case.** `getMostExpensiveAnimal()` must return `null` when the sanctuary has no animals. Do not let it throw `IndexOutOfBoundsException`.
43. **Static `nextId` resets between runs.** The IDs auto-increment across all animals created in a single program run. If you create Ruby first, she is #001. If you create Flash seventh, he is #007. This is shared state via the `static` keyword.

Bonus goals

These are not required for full marks. They are for students who finish early and want a challenge.

44. **Comparable ordering.** Implement `Comparable<Animal>` in `Animal`, with natural ordering by species ascending, then by nickname ascending. Add `getSortedRoster()` to `Sanctuary` that returns a sorted copy of the animal list using `Collections.sort()`.
45. **Endangered report.** Add `getEndangeredReport()` to `Sanctuary`. It returns a formatted string listing all animals with "Critical" health status that have fewer than 3 sightings. Include a header line and the count.
46. **Multi-sanctuary GUI.** Modify `SanctuaryGUI` to accept an `ArrayList<Sanctuary>` and add a second `JComboBox` that lets the user switch between sanctuaries. When the selection changes, the search runs against the newly selected sanctuary.
47. **Find by ID.** Add `findAnimalById(int id)` to `Sanctuary`. Loop through the list, use `equals` to check, and return the matching animal or `null`. This demonstrates why `equals` and `hashCode` matter for object identity.